

So sánh Bloom Filter và Counting Bloom Filter trong bài toán lọc dữ liệu tốc độ cao

Báo cáo học thuật

Ngày 10 tháng 8 năm 2026

Tóm tắt nội dung

Báo cáo so sánh Bloom Filter (BF, Bloom 1970) và Counting Bloom Filter (CBF, Fan *et al.* 2000) cho bài toán lọc dữ liệu tốc độ cao (lọc gói tin, deep packet inspection, lọc truy vấn cơ sở dữ liệu, lọc cache, khử trùng lặp). Chúng tôi trình bày cơ sở lý thuyết, công thức xác suất dương tính giả (false positive), phân tích tràn counter, mã giả cho các thao tác insert/query/delete, và một bộ thực nghiệm Python tự cài đặt cả hai cấu trúc. Kết quả trên $n = 100\,000$ khóa xác nhận: (i) tỉ lệ dương tính giả (FPR) thực nghiệm bám sát công thức $p = (1 - e^{-kn/m})^k$ và khi dùng cùng bộ hàm băm, cùng m , k thì BF và CBF cho kết quả truy vấn *trùng nhau từng khóa*; (ii) CBF 4-bit tốn đúng $4\times$ bộ nhớ BF, do đó với cùng ngân sách byte, FPR của CBF (30.1%) cao hơn BF (0.82%) khoảng 37 lần — hơn một bậc độ lớn; (iii) CBF hỗ trợ xóa nhưng việc xóa phần tử bị nhận nhầm (false positive) tạo ra 8063 false negative (8.06%) — loại lỗi mà BF không bao giờ mắc. Kết luận: dùng BF cho tập gần như tĩnh; chỉ dùng CBF khi bắt buộc phải xóa động; khi cần xóa với FPR mục tiêu $< 3\%$, Cuckoo Filter là lựa chọn đáng cân nhắc hơn.

1 Giới thiệu

Lọc dữ liệu tốc độ cao quy về bài toán *kiểm tra thành viên tập hợp xấp xỉ* (approximate set-membership): cho tập S và khóa x (gói tin, chữ ký mã độc, khóa cơ sở dữ liệu, fingerprint khối dữ liệu), cần trả lời nhanh “ x chắc chắn không thuộc S ” hoặc “ x có thể thuộc S ” với ngân sách bộ nhớ đủ nhỏ để nằm gọn trong SRAM/cache. Việc chấp nhận một tỉ lệ nhỏ dương tính giả cho phép giảm mạnh không gian lưu trữ so với bảng băm chính xác — đây là đánh đổi không gian/thời gian kinh điển của Bloom [1].

Các miền ứng dụng tiêu biểu gồm: (i) *lọc gói tin và DPI/IDS*: đối sánh chữ ký Snort bằng Bloom filter song song trên FPGA [8]; (ii) *lọc truy vấn cơ sở dữ liệu* kiểu LSM-tree: Cassandra, RocksDB, HBase gắn một BF cho mỗi SSTable để bỏ qua các tệp “chắc chắn không chứa khóa”, tránh I/O đĩa; (iii) *web cache hợp tác*: Cache Digest của Squid và giao thức Summary Cache [2]; (iv) *khử trùng lặp* (deduplication) trên fingerprint khối dữ liệu.

Đóng góp của báo cáo: hệ thống hóa lý thuyết BF/CBF; cài đặt độc lập cả hai cấu trúc (kể cả CBF 4-bit đóng gói nibble đúng nghĩa); và bộ thực nghiệm định lượng bốn khía cạnh: độ chính xác, thông lượng, bộ nhớ, và rủi ro false negative khi xóa.

2 Cơ sở lý thuyết

2.1 Bloom Filter (Bloom, 1970)

BF gồm mảng B có m bit khởi tạo 0 và k hàm băm độc lập, phân bố đều $h_1, \dots, h_k : \mathcal{U} \rightarrow \{0, \dots, m-1\}$. *Insert*(x) đặt $B[h_i(x)] = 1$ với mọi i ; *Query*(x) trả “có thể có” nếu mọi bit tương ứng bằng 1, ngược lại trả “chắc chắn không”. Bit đã bật không bao giờ tự tắt nên BF **không có false negative**, nhưng **có false positive** do va chạm băm. BF **không hỗ trợ xóa**: tắt các bit của x có thể tắt nhầm bit đang được phần tử khác dùng chung.

2.2 Counting Bloom Filter (Fan *et al.*, 2000)

CBF thay mỗi bit bằng một *counter* b bit (thường $b = 4$), ký hiệu $C[i]$. *Insert* tăng, *Delete* giảm k counter tương ứng; *Query* trả “có thể có” khi mọi counter dương. CBF hỗ trợ xóa vì mỗi counter đếm số phần tử băm vào ô đó; đổi lại phát sinh hai rủi ro: *trần counter* khi hơn $2^b - 1$ phần tử băm vào cùng một ô, và *false negative* nếu dùng counter bão hòa hoặc xóa sai (xóa phần tử chưa từng chèn nhưng bị báo nhầm là “có”) [2, 3].

2.3 Công thức xác suất

Sau khi chèn n phần tử với k hàm băm, xác suất một bit cụ thể còn 0:

$$p_0 = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}. \quad (1)$$

Xác suất dương tính giả (cả k bit của một khóa lạ đều bằng 1):

$$p \approx \left(1 - e^{-kn/m}\right)^k. \quad (2)$$

Cực tiểu hóa p theo k (tương đương yêu cầu tỉ lệ bit 0 bằng $1/2$ — entropy cực đại) cho số hàm băm tối ưu và bộ nhớ tối ưu theo FPR mục tiêu p :

$$k^* = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}, \quad m = -\frac{n \ln p}{(\ln 2)^2} \iff \frac{m}{n} \approx 1.44 \log_2 \frac{1}{p}. \quad (3)$$

Tại k^* : $p = 2^{-(m/n) \ln 2} \approx 0.6185^{m/n}$. Quy tắc thực dụng: $m/n = 8$ cho FPR $\approx 2\%$; $m/n = 10$ cho FPR $< 1\%$.

Trần counter và vì sao 4 bit là đủ. Số phần tử băm vào một counter tuân theo phân bố nhị thức $B(nk, 1/m) \approx \text{Poisson}(\lambda)$ với $\lambda = kn/m = \ln 2$ ở cấu hình tối ưu. Do đó

$$\Pr[C[i] = 16] \approx \frac{e^{-\ln 2} (\ln 2)^{16}}{16!} \approx 6.78 \times 10^{-17}, \quad (4)$$

và Fan *et al.* chặn trên toàn cục: với $k \leq (\ln 2)m/n$, $\Pr[\max_i C[i] \geq 16] \leq m \cdot 1.37 \times 10^{-15}$ [2]. Vì xác suất này “minuscule”, counter 4 bit (đếm 0..15, bão hòa ở 15) đủ cho hầu hết ứng dụng — CBF tốn $4\times$ bộ nhớ so với BF cùng số ô [3].

Hai hệ quả cho việc so sánh. (1) Với cùng m , k và cùng bộ hàm băm, một ô của CBF dương khi và chỉ khi bit tương ứng của BF bằng 1 (khi chỉ chèn, chưa xóa), nên hai cấu trúc trả lời truy vấn *giống hệt nhau từng khóa* — CBF không “chính xác hơn” BF. (2) Với cùng *ngân sách byte*, BF có số ô gấp b lần CBF b -bit, nên FPR của BF luôn thấp hơn hẳn (Mục 6.5).

3 Mã giả

Thuật toán 1 Bloom Filter: Insert và Query

```
1: procedure BF-INSERT( $x$ )
2:   for  $i \leftarrow 1$  to  $k$  do
3:      $B[h_i(x)] \leftarrow 1$ 
4:   end for
5: end procedure
6: procedure BF-QUERY( $x$ )
7:   for  $i \leftarrow 1$  to  $k$  do
8:     if  $B[h_i(x)] = 0$  then
9:       return FALSE ▷ chắc chắn không
10:    end if
11:  end for
12:  return TRUE ▷ có thể có
13: end procedure
```

Thuật toán 2 Counting Bloom Filter: Insert, Query, Delete (counter bão hòa)

```
1: procedure CBF-INSERT( $x$ )
2:   for  $i \leftarrow 1$  to  $k$  do
3:     if  $C[h_i(x)] < 2^b - 1$  then
4:        $C[h_i(x)] \leftarrow C[h_i(x)] + 1$ 
5:     end if ▷ bão hòa ở  $2^b - 1$ 
6:   end for
7: end procedure
8: procedure CBF-QUERY( $x$ )
9:   for  $i \leftarrow 1$  to  $k$  do
10:    if  $C[h_i(x)] = 0$  then
11:      return FALSE
12:    end if
13:  end for
14:  return TRUE
15: end procedure
16: procedure CBF-DELETE( $x$ )
17:   if CBF-QUERY( $x$ ) then ▷ chỉ xóa nếu được xem là có mặt
18:     for  $i \leftarrow 1$  to  $k$  do
19:       if  $0 < C[h_i(x)] < 2^b - 1$  then
20:          $C[h_i(x)] \leftarrow C[h_i(x)] - 1$ 
21:       end if ▷ không giảm counter đã bão hòa
22:     end for
23:   end if
24: end procedure
```

4 So sánh định tính

Bảng 1: So sánh Bloom Filter và Counting Bloom Filter

Tiêu chí	Bloom Filter	Counting Bloom Filter
Đơn vị ô	1 bit	counter b bit ($b=4$)
Bộ nhớ	m bit	$b \cdot m$ bit ($\sim 4\times$)
Xóa (delete)	Không	Có
False positive	Có, $p \approx (1 - e^{-kn/m})^k$	Có (cùng công thức)
False negative	Không bao giờ	Có thể (tràn/xóa sai)
Insert/Query	$O(k)$	$O(k)$
FPR cùng ngân sách byte	Thấp hơn ($b \times$ số ô)	Cao hơn
Song song hóa cập nhật	Ghi 1 bit an toàn	Đọc-sửa-ghi cần đồng bộ
Hợp phần cứng	Rất tốt (SRAM on-chip)	Tốn SRAM hơn (thường ở phần mềm)
Ứng dụng điển hình	SSTable filter, Cache Digest, chữ ký DPI tĩnh, dedup	Router xóa flow, cache động, Summary Cache cục bộ

5 Phương pháp thực nghiệm

Môi trường. Python 3.14.4, CPU Intel(R) Core(TM) i7-10700K CPU @ 3.80GHz; các gói `mmh3` (MurmurHash3), `bitarray`, `matplotlib`, `numpy` và `pandas`. Toàn bộ mã thực nghiệm nằm trong notebook `thuc_nghiem.ipynb`; chạy hết 20 giây.

Cài đặt. Ba cấu trúc được cài đặt từ đầu (notebook `thuc_nghiem.ipynb`): **BF** dùng mảng bit (`bitarray`); **CBF 4-bit** đóng gói 2 counter/byte (nibble packing) — đúng chi phí $4\times$ như phân tích lý thuyết; **CBF 8-bit** dùng 1 byte/counter (cách cài đặt đơn giản thường gặp, chi phí $8\times$). Cả ba dùng chung sơ đồ double hashing Kirsch–Mitzenmacher $g_i(x) = (h_1(x) + i h_2(x)) \bmod m$ với h_1, h_2 là MurmurHash3 hai seed khác nhau [5], nhờ đó BF và CBF thăm *cùng các vị trí* với mỗi khóa — cô lập đúng biến số cần so sánh. Delete của CBF có kiểm tra Query trước (chỉ xóa khi phần tử được xem là có mặt) và không giảm counter đã bão hòa.

Dữ liệu và tham số. $n = 100\,000$ khóa “có mặt” (P000000000...) được chèn và $n' = 100\,000$ khóa “vắng mặt” (A000000000...) dùng đo FPR; sinh tất định để tái lập được. Tỷ lệ $m/n \in \{4, 6, 8, 10, 12, 16\}$; $k = \text{round}((m/n) \ln 2)$ trừ khi quét k .

Bốn thực nghiệm.

- E1** FPR thực nghiệm vs. lý thuyết và thông lượng insert/query/delete theo m/n .
- E2** FPR theo $k \in \{1, \dots, 12\}$ tại $m/n = 10$ — kiểm chứng $k^* = (m/n) \ln 2$.
- E3** Cố định ngân sách 125 000 byte, so FPR của BF (1 bit/ô), CBF 4-bit và CBF 8-bit.
- E4** Minh họa false negative: xóa các khóa vắng mặt bị CBF báo nhầm “có”, rồi đếm số phần tử thật bị mất.

Giả thuyết. **H1:** FPR thực nghiệm của BF và CBF (cùng m, k) trùng nhau và bám đường lý thuyết. **H2:** bộ nhớ CBF 4-bit = $4\times$ BF (8-bit = $8\times$). **H3:** thông lượng BF $\geq$ CBF; CBF thêm chi phí delete. **H4:** FPR giảm mũ theo m/n ; cực tiểu theo k đạt quanh k^* .

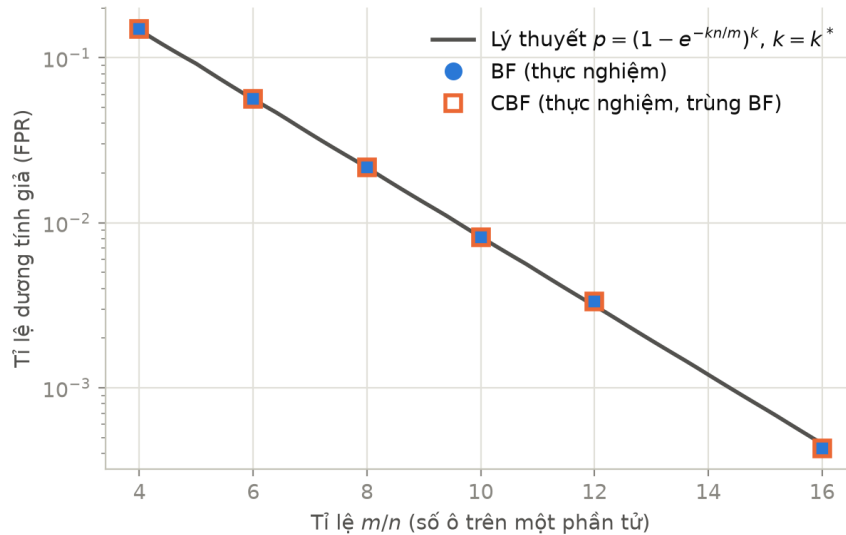
6 Kết quả và thảo luận

6.1 FPR thực nghiệm bám sát lý thuyết (H1, H4)

Bảng 2 và Hình 1 cho thấy FPR thực nghiệm bám sát công thức (2) trên toàn dải m/n : tại $m/n = 8$ lý thuyết 2.16% so với đo được 2.16%; tại $m/n = 10$ lý thuyết 0.82% so với 0.82%. Tỷ lệ bit 1 đo được tại $m/n = 10$ là 0.503 (lý thuyết 0.503, xấp xỉ mức 1/2 entropy cực đại). Đúng như phân tích ở Mục 2.3, cột FPR của BF và CBF *trùng nhau từng giá trị* (được kiểm tra bằng `assert` trên từng khóa): khi chỉ chèn, CBF chính là BF nhìn qua phép thử “counter dương”.

Bảng 2: E1 — FPR lý thuyết vs. thực nghiệm ($n = 100\,000$, $k = \text{round}((m/n) \ln 2)$)

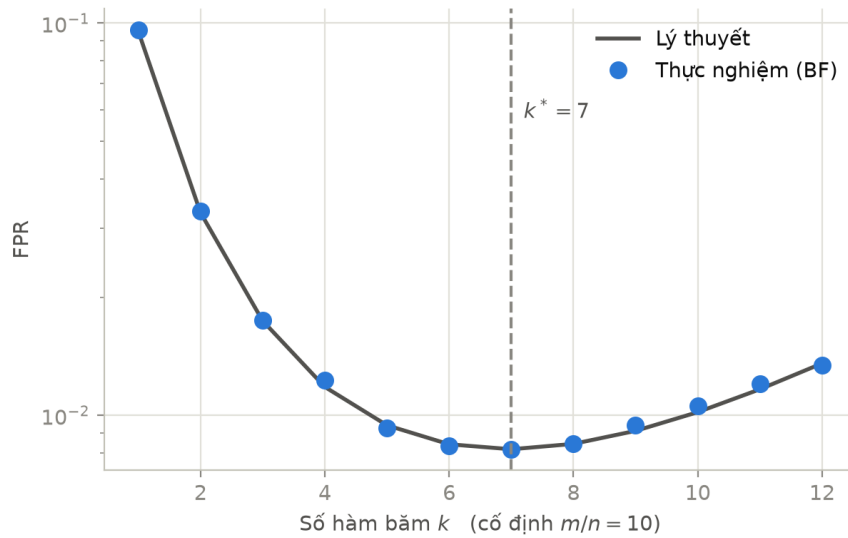
m/n	k	FPR lý thuyết	FPR BF	FPR CBF	Tỉ lệ bit 1 (đo)	(lý thuyết)
4	3	14.7%	14.9%	14.9%	0.529	0.528
6	4	5.606%	5.650%	5.650%	0.487	0.487
8	6	2.158%	2.164%	2.164%	0.528	0.528
10	7	0.819%	0.819%	0.819%	0.503	0.503
12	8	0.314%	0.334%	0.334%	0.487	0.487
16	11	0.046%	0.043%	0.043%	0.497	0.497



Hình 1: FPR theo m/n (trục tung log). Điểm thực nghiệm của BF và CBF trùng khít lên đường lý thuyết $p = (1 - e^{-kn/m})^k$ tại $k = k^*$.

6.2 Số hàm băm tối ưu (H4)

Hình 2 quét k từ 1 đến 12 tại $m/n = 10$: FPR đạt cực tiểu quanh $k^* = 7$ đúng như công thức (3). Cơ chế: k quá nhỏ cho một khóa lạ quá ít phép thử độc lập nên hiếm cơ hội bắt gặp được một bit 0; k quá lớn làm đầy mảng bit khiến từng phép thử dễ trùng bit 1 hơn.



Hình 2: FPR theo số hàm băm k tại $m/n = 10$: cực tiểu tại $k^* = (m/n) \ln 2 \approx 7$.

6.3 Bộ nhớ: CBF trả giá $4\times$ (H2)

Bảng 3 xác nhận chi phí bộ nhớ đo thực tế: CBF 4-bit (nibble packing) tốn đúng $4.0\times$ BF ở mọi cấu hình; bản cài đặt đơn giản bằng byte (uint8) tốn $8\times$. Tại $m/n = 10$: BF dùng 125 000 byte, CBF 4-bit 500 000 byte, CBF 8-bit 1 000 000 byte.

Bảng 3: E1 — Bộ nhớ cấp phát thực tế theo m/n ($n = 100\,000$)

m/n	m (ô)	BF (byte)	CBF 4-bit (byte)	CBF 8-bit (byte)	CBF4/BF
4	400 000	50 000	200 000	400 000	$4.0\times$
6	600 000	75 000	300 000	600 000	$4.0\times$
8	800 000	100 000	400 000	800 000	$4.0\times$
10	1 000 000	125 000	500 000	1 000 000	$4.0\times$
12	1 200 000	150 000	600 000	1 200 000	$4.0\times$
16	1 600 000	200 000	800 000	1 600 000	$4.0\times$

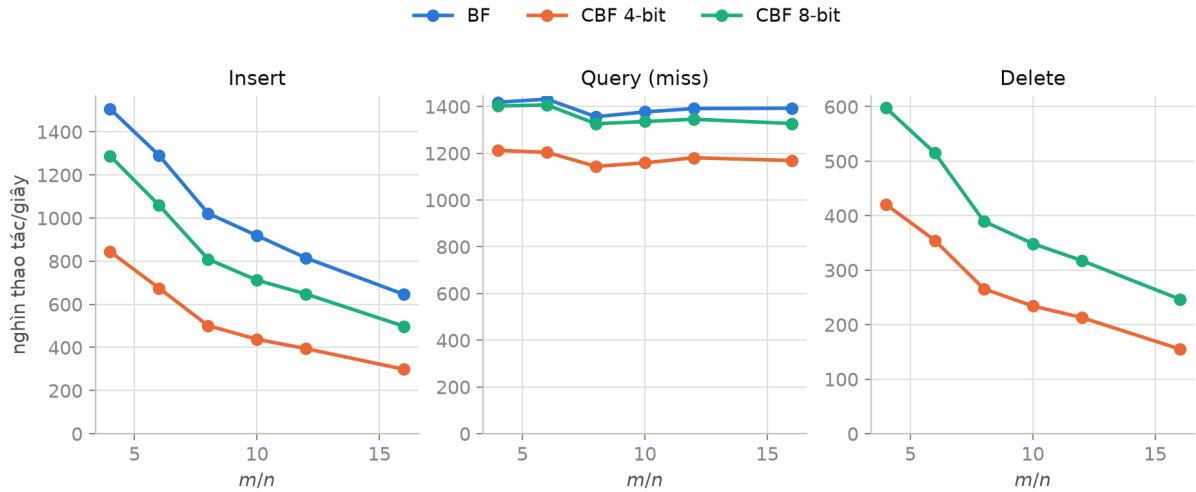
6.4 Thông lượng (H3)

Bảng 4: E1 — Thông lượng (nghìn thao tác/giây; CBF4 = CBF 4-bit)

m/n	k	Insert		Query (miss)		Query (hit) / Delete	
		BF	CBF4	BF	CBF4	BF hit	CBF4 del
4	3	1 507	844	1 419	1 212	1 181	421
6	4	1 291	675	1 432	1 204	1 047	355
8	6	1 022	501	1 357	1 144	842	266
10	7	919	438	1 377	1 159	748	234
12	8	815	395	1 391	1 181	695	213
16	11	646	299	1 393	1 169	566	155

Tại $m/n = 10$ (Bảng 4, Hình 3): BF đạt 919 nghìn insert/s so với 438 của CBF 4-bit (nhanh gấp 2.10 lần) — CBF phải đọc-sửa-ghi nibble thay vì chỉ bật bit. Với query trên khóa vắng

mặt, BF đạt 1 377 nghìn thao tác/s so với 1 159 của CBF (gấp 1.19 lần). Query trên khóa vắng mặt (miss) nhanh hơn trên khóa có mặt (hit): với BF là 1 377 so với 748 nghìn thao tác/s, vì truy vấn miss thoát sớm ngay khi gặp bit 0 đầu tiên, còn truy vấn hit luôn phải thăm đủ k vị trí. Delete của CBF 4-bit chỉ đạt 234 nghìn thao tác/s — đắt nhất trong các thao tác vì gồm một lượt Query đầy đủ cộng k lần đọc–sửa–ghi. Lưu ý các con số tuyệt đối phản ánh chi phí thông dịch của Python; điều có ý nghĩa là *tỉ số tương đối* giữa các cấu trúc, còn cài đặt C/C++ sẽ nhanh hơn 1–2 bậc độ lớn.



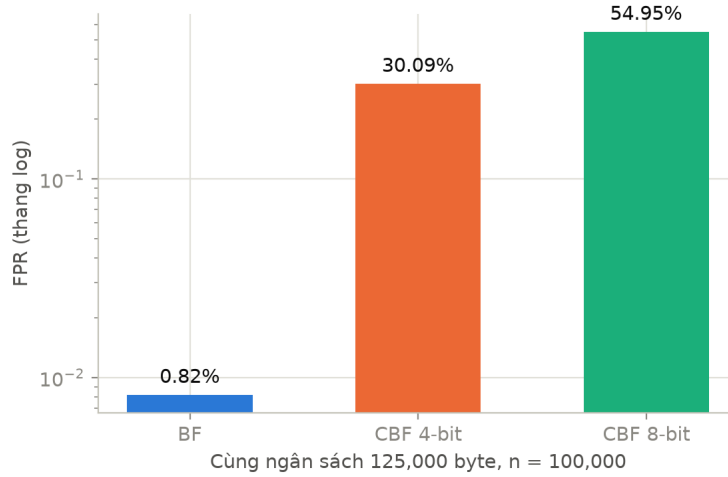
Hình 3: Thông lượng insert, query (miss) và delete theo m/n .

6.5 Cùng ngân sách bộ nhớ: BF thắng áp đảo

Đây là góc so sánh công bằng nhất khi bộ nhớ là ràng buộc cứng (SRAM on-chip). Với cùng 125 000 byte (Bảng 5, Hình 4): BF có 1 000 000 ô nên FPR chỉ 0.82%; CBF 4-bit còn 250 000 ô, FPR vọt lên 30.1%; CBF 8-bit còn 125 000 ô, FPR 54.9%. Khoảng cách 37–67 lần (từ hơn một tới gần hai bậc độ lớn) là cái giá thật sự của khả năng xóa.

Bảng 5: E3 — Cùng ngân sách 125 000 byte, $n = 100\,000$, k tối ưu cho từng cấu trúc

Cấu trúc	Số ô	bit/ô	k	FPR lý thuyết	FPR thực nghiệm
BF	1 000 000	1	7	0.819%	0.819%
CBF 4-bit	250 000	4	2	30.3%	30.1%
CBF 8-bit	125 000	8	1	55.1%	54.9%



Hình 4: FPR (thang log) khi cả ba cấu trúc dùng cùng một lượng byte.

6.6 False negative của CBF khi xóa

Thực nghiệm E4 (Bảng 6) tái hiện kịch bản xóa sai trong thực tế: một hệ thống tin vào câu trả lời “có” của CBF và đem xóa 2164 khóa *chưa từng được chèn* (chúng là false positive tại $m/n = 8$); trong đó 2041 lần xóa thực sự được thực hiện — số còn lại bị chính phép kiểm tra query trong CBF-DELETE chặn lại, vì các lần xóa trước đó đã làm giảm counter liên quan. Mỗi lần xóa thành công giảm nhằm tới $k = 6$ counter đang được các phần tử thật dùng chung, kết quả 8063 phần tử thật (8.06%) biến mất khỏi bộ lọc — false negative, loại lỗi phá vỡ bất biến “không bỏ sót” mà mọi ứng dụng của BF dựa vào. BF không thể mắc lỗi này vì không có thao tác xóa. Biện pháp thực dụng: xây lại (rebuild) bộ lọc định kỳ, hoặc chỉ xóa các khóa chắc chắn đã chèn (cần cấu trúc phụ trợ).

Bảng 6: E4 — Xóa các khóa bị báo nhầm “có” gây false negative trong CBF	
Dại lượng	Giá trị
Cấu hình	$m/n = 8, k = 6, n = 100\,000$
Khóa vắng mặt bị báo nhầm “có” (false positive)	2164
Lần xóa thực sự thực hiện (delete có kiểm tra)	2041
Phần tử thật bị mất (false negative)	8063
Tỉ lệ false negative	8.063%

7 Các biến thể liên quan

Blocked Bloom Filter [4]: gói toàn bộ k bit của một khóa vào một block bằng đúng một cache line, giảm số cache miss mỗi truy vấn xuống ~ 1 , đổi lấy FPR tăng nhẹ; RocksDB dùng chiến lược này; không hỗ trợ xóa. **Cuckoo Filter** [7]: lưu fingerprint trong bảng băm cuckoo, *hỗ trợ xóa*, và cần ít không gian hơn cả BF tối ưu khi FPR mục tiêu $\varepsilon < 3\%$; nhược điểm là chèn có thể thất bại khi hệ số tải vượt $\sim 95\%$. **d-left CBF** [6]: dùng d-left hashing + fingerprint, tiết kiệm khoảng một nửa không gian so với CBF 4-bit — lựa chọn thay thế CBF tốt trên phần cứng.

8 Khuyến nghị lựa chọn

1. **Tập gần như tĩnh hoặc rebuild được định kỳ** (SSTable bất biến, Cache Digest, chữ ký DPI ít đổi): dùng **BF**; chọn $m = -n \ln p / (\ln 2)^2$ và $k = \text{round}((m/n) \ln 2)$ theo FPR

mục tiêu p (ví dụ $p = 1\% \Rightarrow m/n \approx 9.6, k = 7$).

2. **Bắt buộc xóa động, FPR mục tiêu $\geq 3\%$** hoặc cần đếm bội: dùng **CBF 4-bit** với counter bão hòa; lên lịch rebuild định kỳ để triệt tích lũy false negative; nếu có thể, chỉ xóa khóa chắc chắn đã chèn.
3. **Bắt buộc xóa, FPR mục tiêu $< 3\%$** : cân nhắc **Cuckoo Filter** (ít không gian hơn, lookup nhanh, có xóa) hoặc **dlCBF** trên phần cứng; theo dõi hệ số tải.
4. **Nghẽn vì cache miss**: chuyển sang **Blocked Bloom Filter**, chấp nhận FPR tăng nhẹ.

9 Kết luận

Thực nghiệm xác nhận bốn giả thuyết: FPR của BF/CBF trùng nhau và bám công thức lý thuyết (H1), CBF 4-bit tốn đúng $4\times$ bộ nhớ (H2), BF nhanh hơn CBF 4-bit ở mọi thao tác chung — riêng CBF 8-bit bám sát BF ở query nhưng vẫn chậm hơn ở insert — và CBF gánh thêm chi phí delete (H3), FPR giảm mũ theo m/n với cực tiểu theo k tại k^* (H4). Bức tranh tổng thể: *CBF không thay thế BF — nó mua khả năng xóa bằng bộ nhớ gấp bốn và rủi ro false negative*. Do đó BF là mặc định đúng cho lọc dữ liệu tốc độ cao trên tập tĩnh; CBF chỉ dành cho tập biến động liên tục không thể rebuild; và khi cần xóa với yêu cầu FPR chặt, các cấu trúc thế hệ sau như Cuckoo Filter đáng được ưu tiên khảo sát.

Hạn chế. Số đo thông lượng mang chi phí thông dịch của Python nên chỉ có ý nghĩa so sánh tương đối; công thức (2) là xấp xỉ (và là cận dưới) — với m nhỏ nên đo thực nghiệm; phân tích giả định hàm băm lý tưởng, còn double hashing chỉ hợp lệ tiệm cận và trong môi trường đối kháng cần hàm băm có khóa.

Tài liệu

- [1] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [2] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, “Summary Cache: A Scalable Wide-Area Web Cache Sharing Protocol,” *IEEE/ACM Transactions on Networking*, vol. 8, no. 3, pp. 281–293, 2000.
- [3] A. Broder and M. Mitzenmacher, “Network Applications of Bloom Filters: A Survey,” *Internet Mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [4] F. Putze, P. Sanders, and J. Singler, “Cache-, hash-, and space-efficient Bloom filters,” *ACM Journal of Experimental Algorithmics*, vol. 14, 2010.
- [5] A. Kirsch and M. Mitzenmacher, “Less hashing, same performance: Building a better Bloom filter,” *Random Structures & Algorithms*, vol. 33, no. 2, pp. 187–218, 2008.
- [6] F. Bonomi, M. Mitzenmacher, R. Panigrahy, S. Singh, and G. Varghese, “An Improved Construction for Counting Bloom Filters,” in *Algorithms — ESA 2006*, LNCS 4168, pp. 684–695.
- [7] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, “Cuckoo Filter: Practically Better Than Bloom,” in *Proc. 10th ACM CoNEXT*, pp. 75–88, 2014.
- [8] S. Dharmapurikar, P. Krishnamurthy, T. Sproull, and J. Lockwood, “Deep Packet Inspection using Parallel Bloom Filters,” in *IEEE Hot Interconnects 11*, 2003.